

Computational Data Analysis (02582)

SVM

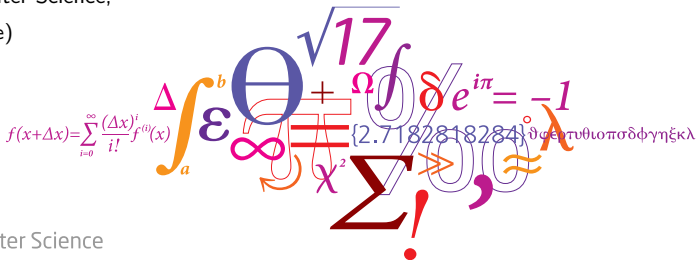
Sneha Das

Assistant Professor

Department of Applied Mathematics and Computer Science,

Technical University of Denmark (DTU Compute)

(Slides adapted from Line Clemmensen)



DTU Compute

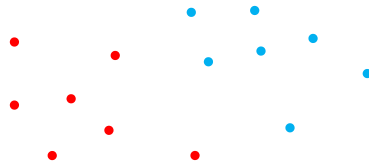
Department of Applied Mathematics and Computer Science

Outline I

- Part I: Support vector machines
 - Introducing the margin
 - Properties of SVMs
- APPENDIX

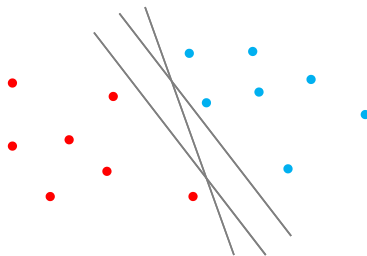
$$y_{\text{new}} = \text{sign}(\beta_0 + x_{\text{new}}\beta)$$

- For practical reasons, we label the two classes 1 and -1 .



The decision function

- Many hyperplanes can separate the two classes
- What would be optimal??!

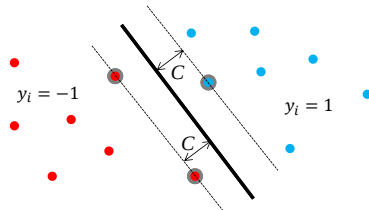


Part I: Support vector machines

Introducing the 'margin'

Maximize the distance C (margin) from the decision line to the nearest points in each class.

There is no probabilistic model here as we have for linear discriminant analysis and logistic regression



Unpacking the Math: The Two Pillars of SVM

At its core, the Support Vector Machine is the result of two powerful ideas:

1. The Optimal Separating Hyperplane (OSH)

- Focuses on finding the unique boundary that maximizes the distance to the nearest points.
- Solved via **Quadratic Programming**.
- Result: **Sparsity** (The model only 'sees' Support Vectors).

2. Feature Space Transformation

- Moving data from $\mathbb{R}^d$ to a higher-dimensional space $\mathcal{H}$ where linear separation is possible.
- Calculated implicitly via the **Kernel Trick**.
- Result: **Non-linear boundaries** for the price of linear ones.

$$\text{SVM} = \text{Convex Optimization} + \text{Implicit Feature Mapping}$$

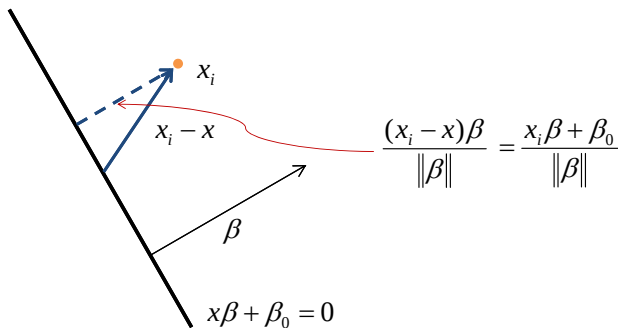
Optimal Separating Hyperplanes (OSH)

We wish to maximize the margin between classes.

To maximize the margin, we first need a mathematical expression for the distance from a data point x_i to the decision boundary (hyperplane).

The Setup:

- Let the hyperplane be defined by: $x^T \beta + \beta_0 = 0$
- Let x_i be an arbitrary point in space.
- Let x be a point that lies exactly on the hyperplane.



Step 1: The Normal Vector

The vector β is **orthogonal** (perpendicular) to the hyperplane.

To find the distance, we need a **unit normal vector** (a vector in the same direction but with length 1). We define this as:

$$\hat{n} = \frac{\beta}{\|\beta\|}$$

This vector $\hat{n}$ gives us the 'direction' of the shortest path to the plane.

Step 2: The Projection Vector

Consider the vector $\vec{v}$ that connects a point on the plane (x) to our data point (x_i):

$$\vec{v} = (x_i - x)$$

The shortest distance d is the **projection** of this vector $\vec{v}$ onto our unit normal vector $\hat{n}$.

Mathematically, this is the dot product:

$$d = \langle (x_i - x), \hat{n} \rangle = \frac{(x_i - x)^T \beta}{\|\beta\|}$$

Step 3: The Algebraic Expansion

Expanding the dot product from the previous step:

$$d = \frac{x_i^T \beta - x^T \beta}{\|\beta\|}$$

The Key Insight: Because x lies exactly on the hyperplane, we know it satisfies the plane equation:

$$x^T \beta + \beta_0 = 0 \quad \implies \quad -x^T \beta = \beta_0$$

Step 4: The Final Result

Substituting $-x^T \beta = \beta_0$ back into our distance equation, we get the standard expression used in SVM:

Point-to-Plane Distance

$$d = \frac{x_i^T \beta + \beta_0}{\|\beta\|}$$

Note:

- If $d > 0$, the point is on the 'positive' side of the plane.
- If $d < 0$, the point is on the 'negative' side.
- SVM maximizes the **signed distance**: $y_i \frac{x_i^T \beta + \beta_0}{\|\beta\|}; y \in \{-1, 1\}$.

Part I: Support vector machines

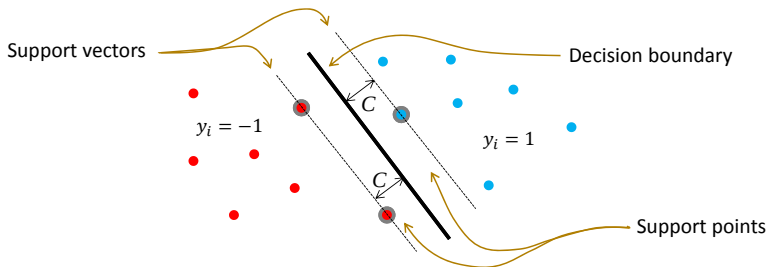
OSH as a maximization problem

We can now formulate a maximization problem

$$\arg \max_{\beta, \beta_0} C$$

such that

$$y_i \frac{x_i \beta + \beta_0}{\|\beta\|} \geq C \quad \forall i$$



Part I: Support vector machines

Solving the OSH problem

- Maximization problem can be turned into a minimization problem

$$\left\{ \begin{array}{l} \arg \min_{\beta, \beta_0} \frac{1}{2} \|\beta\|^2 \\ \text{such that} \\ y_i(x_i\beta + \beta_0) \geq 1 \quad \forall i \end{array} \right.$$

- This is a nonlinear problem with **linear** constraints
 - You could use a function for constrained optimization of **any** nonlinear function.
 - But this one is quadratic (convex) - does this simplify things?

Part I: Support vector machines

Solving the OSH problem

- Maximization problem can be turned into a minimization problem

$$\left\{ \begin{array}{l} \arg \min_{\beta, \beta_0} \frac{1}{2} \|\beta\|^2 \\ \text{such that} \\ y_i(x_i\beta + \beta_0) \geq 1 \quad \forall i \end{array} \right.$$

- This is a nonlinear problem with **linear** constraints
 - You could use a function for constrained optimization of **any** nonlinear function.
 - But this one is quadratic (convex) - does this simplify things?
- Quadratic programming
 - Very efficient solvers exists
 - Python: QP in CVXOPT

Step 1

A hyperplane is defined by the equation:

$$x^T \beta + \beta_0 = 0$$

Scale Invariance: If we multiply the entire equation by a constant $\kappa \neq 0$, the decision boundary **does not move**:

$$\kappa(x^T \beta + \beta_0) = 0 \implies x^T(\kappa\beta) + (\kappa\beta_0) = 0$$

The Canonical Hyperplane: We use this freedom to pick a specific scale where the distance to the nearest points (Support Vectors) results in a numerator of exactly 1:

$$|x_i^T \beta + \beta_0| = 1 \quad \text{for Support Vectors}$$

Step 2

Recall our geometric distance formula for any point x_i :

$$\text{Distance} = \frac{y_i(x_i^T \beta + \beta_0)}{\|\beta\|}$$

Under our new **canonical scaling**, the numerator for the closest points is fixed at 1. Therefore, the margin width C becomes:

$$C = \frac{1}{\|\beta\|}$$

To make the 'road' as wide as possible, we need to make $\|\beta\|$ as small as possible.

Part I: Support vector machines

Step 3

We can now transform the maximization problem into a mathematically equivalent minimization problem:

Geometric Goal

$$\begin{aligned} & \max_{\beta, \beta_0} C \\ & \text{subject to } \frac{y_i(x_i^T \beta + \beta_0)}{\|\beta\|} \geq C \end{aligned}$$

 $\implies$

Computational Goal

$$\begin{aligned} & \min_{\beta, \beta_0} \frac{1}{2} \|\beta\|^2 \\ & \text{subject to } y_i(x_i^T \beta + \beta_0) \geq 1 \end{aligned}$$

Why the $\frac{1}{2}$ and the square?

- Minimizing $\|\beta\|$ is the same as minimizing $\|\beta\|^2$.
- The quadratic form is **convex** (easier to solve).
- The $\frac{1}{2}$ cancels out the 2 from the derivative during optimization.

Primal formulation of OSH problem

We have formulated the OSH problem such that we can use efficient standard numerical solvers. **We have**

- A model with one β coefficient for each dimension of x .
- One constraint for each observation x
- An optimal linear separation between classes

What else could we ask for?

Primal vs. Dual: Visualizing the 'Gap'

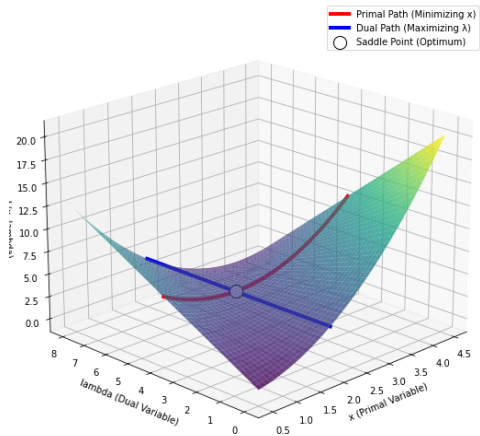
In optimization, we often move between two perspectives.

- **Primal Problem (p^*):** Minimizing the objective directly (The 'View from the Hill').
- **Dual Problem (d^*):** Maximizing the lower bound (The 'View from the Valley').

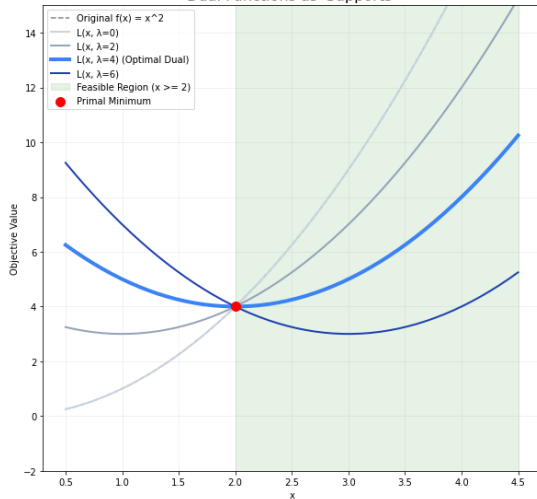
Weak vs. Strong Duality

- **Weak Duality:** $d^* \leq p^*$ always holds. The distance $p^* - d^*$ is the *Duality Gap*.
- **Strong Duality:** $d^* = p^*$. There is **no gap**.

The Lagrangian Saddle Point



Dual Functions as 'Supports'



Dual formulation of OSH problem

Well, it would be nice with

- One coefficient for each observation instead of each dimension.
 - Good idea when we have high-dimensional problems with few observations.
- A non-linear separation between classes.

We can achieve this if we use **Lagrange multipliers**

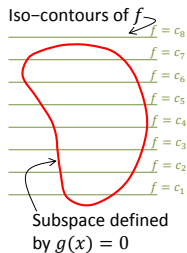
Let us start with a guided tour through the theory of Lagrange's method

We learn to solve $\begin{cases} \max_x f(x) \\ g(x) = 0 \\ h(x) \geq 0 \end{cases}$ using Lagrange multipliers

Part I: Support vector machines

Constrained optimization

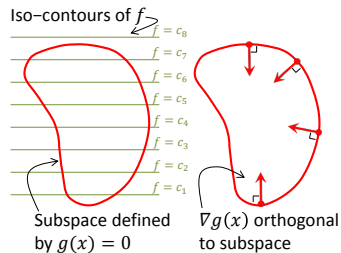
$$\begin{cases} \max_x f(x) \\ g(x) = 0 \end{cases}$$



Part I: Support vector machines

Constrained optimization

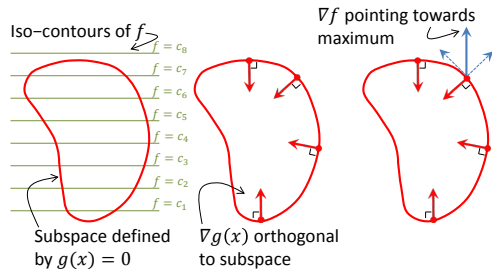
$$\begin{cases} \max_x f(x) \\ g(x) = 0 \end{cases}$$



Part I: Support vector machines

Constrained optimization

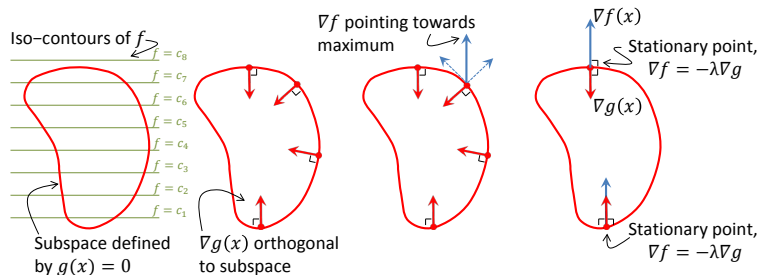
$$\begin{cases} \max_x f(x) \\ g(x) = 0 \end{cases}$$



Part I: Support vector machines

Constrained optimization

$$\begin{cases} \max_x f(x) \\ g(x) = 0 \end{cases}$$



- At the stationary points we can not move along the feasible region ($g(x) = 0$) without decreasing f .
- The stationary points are defined by $\nabla f = -\lambda \nabla g$ for some constant λ

Part I: Support vector machines

Lagrange multipliers

Define the **Lagrange primal function**

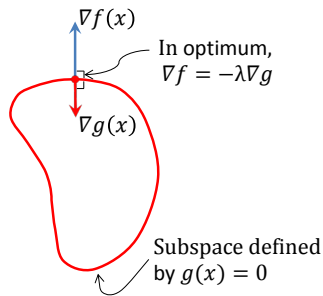
$$L_p(x, \lambda) = f(x) + \lambda g(x)$$

and the **Lagrange multiplier** λ .

Find solution (x^*, λ^*) to

$$\begin{cases} \frac{\partial L_p}{\partial x} = 0 \\ \frac{\partial L_p}{\partial \lambda} = 0 \end{cases} \quad \text{i.e. } \nabla L_p = 0.$$

The stationary points, x^* , might be local maxima, local minima or saddle points.



Lagrange multipliers

A local maximum to the constrained optimization problem

$$\begin{cases} \max_x f(x) \\ g(x) = 0 \end{cases}$$

with Lagrange function

$$L_p(x, \lambda) = f(x) + \lambda g(x)$$

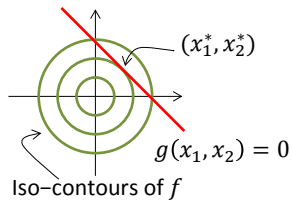
is given by x^* if and only if

- ❶ $\nabla_x L_P(x^*, \lambda^*) = 0$
- ❷ $\nabla_\lambda L_P(x^*, \lambda^*) = 0$
- ❸ Hessian is negative semidefinite

In practice, solve $\nabla L_P = 0$ and verify that the solution is a valid maximum.

Example

$$\begin{cases} \max_x f(x_1, x_2) = 1 - x_1^2 - x_2^2 \\ g(x_1, x_2) = x_1 + x_2 - 1 = 0 \end{cases}$$

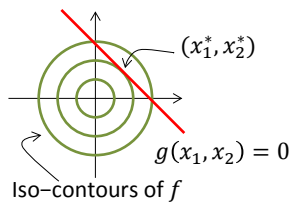


Part I: Support vector machines

Example

$$\begin{cases} \max_x f(x_1, x_2) = 1 - x_1^2 - x_2^2 \\ g(x_1, x_2) = x_1 + x_2 - 1 = 0 \end{cases}$$

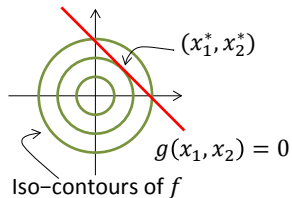
$$L_P(\mathbf{x}, \lambda) = 1 - x_1^2 - x_2^2 + \lambda(x_1 + x_2 - 1)$$



Example

$$\begin{cases} \max_x f(x_1, x_2) = 1 - x_1^2 - x_2^2 \\ g(x_1, x_2) = x_1 + x_2 - 1 = 0 \end{cases}$$

$$L_P(\mathbf{x}, \lambda) = 1 - x_1^2 - x_2^2 + \lambda(x_1 + x_2 - 1)$$



$$\begin{cases} \frac{\partial L_P}{\partial x_1} = -2x_1 + \lambda = 0 \\ \frac{\partial L_P}{\partial x_2} = -2x_2 + \lambda = 0 \\ \frac{\partial L_P}{\partial \lambda} = x_1 + x_2 - 1 = 0 \end{cases}$$

Part I: Support vector machines

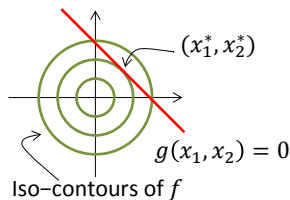
Example

$$\begin{cases} \max_x f(x_1, x_2) = 1 - x_1^2 - x_2^2 \\ g(x_1, x_2) = x_1 + x_2 - 1 = 0 \end{cases}$$

$$L_P(\mathbf{x}, \lambda) = 1 - x_1^2 - x_2^2 + \lambda(x_1 + x_2 - 1)$$

$$\begin{cases} \frac{\partial L_P}{\partial x_1} = -2x_1 + \lambda = 0 \\ \frac{\partial L_P}{\partial x_2} = -2x_2 + \lambda = 0 \\ \frac{\partial L_P}{\partial \lambda} = x_1 + x_2 - 1 = 0 \end{cases}$$

Solution/optimum is at $(x_1^*, x_2^*) = (1/2, 1/2)$ with $\lambda = 1$.



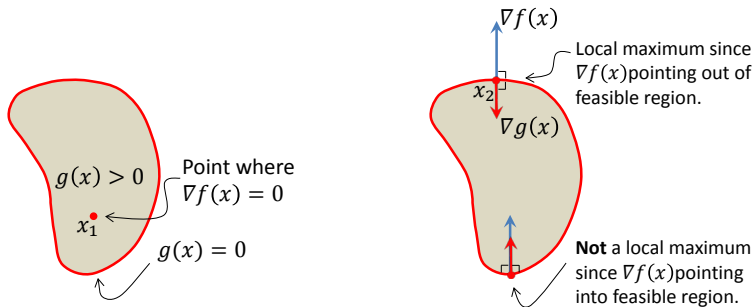
Part I: Support vector machines

Inequalities in constraints, $g(x) \geq 0$

Constrained optimization with inequality constraints

$$\begin{cases} \max_x f(x) \\ g(x) \geq 0 \end{cases}$$

The optimum is either within the feasible region $g(x) > 0$ or along the edge $g(x) = 0$.



Notice that ∇g is always pointing inwards since $g > 0$ in the feasible region and $g = 0$ along the edge.

Part I: Support vector machines

Inequalities in constraints, $g(x) \geq 0$

① Constraint is inactive, i.e. $g(x_1) > 0$.

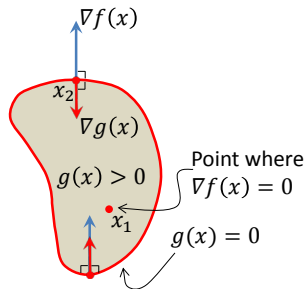
- Solution by $\nabla f(x_1) = 0$, ie Lagrange function,

$$\begin{cases} \nabla f(x_1) = -\lambda \nabla g(x_1) \\ \lambda = 0 \end{cases}$$

② Constraint is active, i.e. $g(x_2) = 0$.

- As before with $\lambda \neq 0$. Important with the sign of λ . In maximum $\nabla f(x_2)$ is pointing out of the region $g(x_2) > 0$, i.e.

$$\begin{cases} \nabla f(x_2) = -\lambda \nabla g(x_2) \\ \lambda > 0 \end{cases}$$



The KKT Conditions: Optimizing with Inequalities

For a local maximum x^* of the problem $\max f(x)$ subject to $g(x) \geq 0$, with Lagrangian:

$$L_P(x, \lambda) = f(x) + \lambda g(x)$$

The following **Karush-Kuhn-Tucker (KKT)** conditions must hold:

The Mathematical Conditions

① $\nabla_x L_P(x^*, \lambda^*) = 0$

② $\lambda^* \geq 0$

③ $\lambda^* g(x^*) = 0$

④ $g(x^*) \geq 0$

5. Hessian $\nabla^2 L_P$ is negative definite

The Physical Meaning

- **Stationarity:** Force balance.
- **Dual Feasibility:** "Pressure" direction.
- **Comp. Slackness:** Sparse logic.
- **Primal Feasibility:** Follow the rules.

The SVM Connection

Condition 3 (**Complementary Slackness**) is the 'Sparsity Engine': It ensures that if a point is safe ($g(x) > 0$), its influence on the model is zero ($\lambda = 0$).

Part I: Support vector machines

Multiple constraints

A local maximum to the constrained optimization problem

$$\begin{cases} \max_x f(x) \\ g_j(x) = 0 & \forall j \\ h_k(x) \geq 0 & \forall k \end{cases}$$

with Lagrange function

$$L_p(x, \lambda, \mu) = f(x) + \sum_j \lambda_j g_j(x) + \sum_k \mu_k h_k(x)$$

is given by x^* if and only if

- ❶ $\nabla_x L_P(x^*, \lambda^*, \mu^*) = 0$
- ❷ $\mu_k^* \geq 0$
- ❸ $\mu_k^* h_k(x^*) = 0$
- ❹ $h_k(x^*) \geq 0$
- ❺ $g_j(x^*) = 0$ (same as $\nabla_\lambda L_P(x^*, \lambda^*, \mu^*) = 0$)
- ❻ Negative definite constraints on Hessian.

Dual problem: The Lagrangian as an Upper Bound

Recall our Primal function with equalities $g(x) = 0$ and inequalities $h(x) \geq 0$:

$$L_P(x, \lambda, \mu) = f(x) + \sum \lambda_j g_j(x) + \sum \mu_k h_k(x)$$

The Logic for a Feasible Point x^* : For any valid multipliers ($\mu_k \geq 0$ and any λ_j), at the optimum:

- ❶ $\lambda_j g_j(x^*) = 0$ (Since the constraint is exactly zero)
- ❷ $\mu_k h_k(x^*) \geq 0$ (Since both μ and h are non-negative)

$$\implies L_P(x^*, \lambda, \mu) = f(x^*) + 0 + (\text{positive term}) \geq f(x^*)$$

Dual problem: Defining the Dual Function

Since $L_P(x^*, \lambda, \mu) \geq f(x^*)$, it follows that the **maximum** of the Lagrangian over *all* x must also be an upper bound:

The Lagrange Dual Function

$$L_D(\lambda, \mu) = \max_x L_P(x, \lambda, \mu) \geq f(x^*)$$

The Dual Optimization Strategy: We want the 'best' (tightest) upper bound possible. This means we must **minimize** the dual function:

$$\min_{\lambda, \mu \geq 0} L_D(\lambda, \mu) = \min_{\lambda, \mu \geq 0} \max_x L_P(x, \lambda, \mu)$$

If Strong Duality holds, the minimum of this dual 'ceiling' meets the maximum of our primal 'floor' exactly.

Slater's condition

Dual problem, solves original problem if equality in

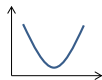
$$\min_{\substack{\lambda \geq 0 \\ \mu}} L_D(\lambda, \mu) \geq f(x^*)$$

Slater's condition

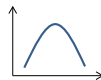
Equality guaranteed if f is concave and constraints are convex

- There must be some x fulfilling all constraints
- Linear constraints are ok
- A local optimum will also be the global optimum.
- No conditions on the Hessian necessary.

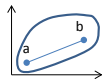
Convex function



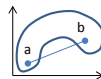
Concave function



Convex set



Concave set



Three formulations

When f is concave and the constraints are linear there are three equal problems:

$$\left\{ \begin{array}{l} \text{Find} \\ \max_x f(x) \\ \text{such that} \\ g(x) = 0 \\ h(x) \geq 0 \end{array} \right. \quad \left\{ \begin{array}{l} \text{Solve} \\ \nabla_x L_P = 0 \\ \text{such that} \\ g(x) = 0 \\ h(x) \geq 0 \\ \mu \geq 0 \\ \mu h(x) = 0 \end{array} \right. \quad \left\{ \begin{array}{l} \text{Find} \\ \min_{\lambda \geq 0, \mu} L_D(\lambda, \mu) \\ \text{such that} \\ \nabla_x L_P = 0 \quad (\text{def. of } L_D) \\ g(x) = 0 \\ h(x) \geq 0 \\ \mu \geq 0 \\ \mu h(x) = 0 \end{array} \right.$$

where

$$L_P(x, \lambda, \mu) = f(x) + \lambda g(x) + \mu h(x)$$

$$L_D(\lambda, \mu) = \max_x L_P(x, \lambda, \mu)$$

Which formulation?

Use the formulation that has

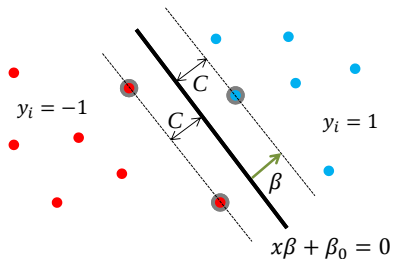
- tractable analytical solution
- efficient numerical solver

Both the primal and the dual function used!

Back to the Optimal Separating Hyperplane

$$\left\{ \begin{array}{l} \arg \max_{\beta, \beta_0} C \\ \text{such that} \\ y_i \frac{(x_i \beta + \beta_0)}{\|\beta\|} \geq C \quad \forall i \end{array} \right.$$

$$\left\{ \begin{array}{l} \arg \min_{\beta, \beta_0} \frac{1}{2} \|\beta\|^2 \\ \text{such that} \\ y_i (x_i \beta + \beta_0) \geq 1 \quad \forall i \end{array} \right.$$



Part I: Support vector machines

Solving the OSH problem

Use the Lagrange multipliers!

Stage 1 Incorporate constraints using Lagrange multipliers

$$\begin{cases} L(\beta, \beta_0, \alpha) = \frac{1}{2} \|\beta\|^2 - \sum_{i=1}^n \alpha_i (y_i (x\beta + \beta_0) - 1) \\ \alpha_i \geq 0 \quad \forall i \end{cases}$$

Stage 2 Differentiate and set to zero

$$\begin{cases} \frac{\partial L}{\partial \beta} = \beta - \sum_i \alpha_i y_i x_i^T = 0 \\ \frac{\partial L}{\partial \beta_0} = \sum \alpha_i y_i = 0 \end{cases}$$

and we have

$$\begin{cases} \beta = \sum_i \alpha_i y_i x_i^T \\ \sum \alpha_i y_i = 0 \end{cases}$$

OSH Dual Formulation: Stage 3

Substituting $\beta = \sum \alpha_i y_i x_i$ back into the Lagrangian yields the **Dual**:

$$\arg \max_{\alpha} \underbrace{\mathbf{1}^T \alpha}_{\sum \alpha_i} - \frac{1}{2} \alpha^T \underbrace{(Y X X^T Y)}_Q \alpha$$

Subject to:

- $\alpha_i \geq 0 \quad \forall i$ (Non-negativity)
- $\sum \alpha_i y_i = 0$ (The β_0 constraint)

The Kernel

The matrix $X X^T$ is the **Gram Matrix** ($G_{ij} = x_i^T x_j$). The optimization depends *only* on the similarity between points!

Mapping to a Standard QP Solver

Most solvers minimize: $\min_{\alpha} \frac{1}{2} \alpha^T Q \alpha + c^T \alpha \quad \text{s.t. } A\alpha \leq b \text{ and } E\alpha = d.$

The SVM Mapping:

- **Q (Hessian matrix):** $Q_{ij} = y_i y_j \langle x_i, x_j \rangle$
- **c (Linear term):** -1 (a vector of -1s, because we flip max to min)
- **A, b (Inequalities):** $A = -I$ and $b = 0$ (to enforce $\alpha_i \geq 0$)
- **E, d (Equalities):** $E = \mathbf{y}^T$ and $d = 0$ (to enforce $\sum \alpha_i y_i = 0$)

How do we get the final model?

- 1 Solve for α using the QP solver.
- 2 Calculate $\beta = \sum \alpha_i y_i x_i$.
- 3 Find β_0 using any Support Vector ($y_i(x_i^T \beta + \beta_0) = 1$).

Two more things...

How do we find β_0 ?

- For the support points we have $y_i(x_i\beta + \beta_0) = 1$
- Use one of the support points to calculate β_0

How do we predict class belonging?

- Along the decision line we have $x\beta + \beta_0 = 0$
- Along the support lines we have $x\beta + \beta_0 = \pm 1$
- Decision based on the side of the line

$$\hat{y}(x_{\text{new}}) = \text{sign}(x_{\text{new}}\beta + \beta_0)$$

Part I: Support vector machines

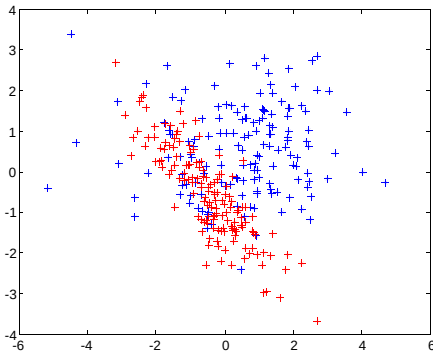
Phew!

- We have found an efficient way of maximizing the margin between classes
- Of course, there are software packages for you!



Wait a minute... Margin??

- For overlapping data, there is no solution
 - What's the use?



Part I: Support vector machines

Basis expansion and Kernels

- We can do OSH on a transformed feature space
- Transformed features gives non-linear decision boundaries.
- With **the Kernel trick** we can use an infinit dimensional feature expansion

Non-linear OSH

Let's try basis expansions!

$$\left\{ \begin{array}{l} \arg \max_{\alpha} \alpha \mathbf{1} - \frac{1}{2} \alpha^T Y X X^T Y \alpha \\ \text{such that} \\ \alpha_i \geq 0 \quad \forall i \\ \sum \alpha_i y_i = 0 \end{array} \right.$$

...turns into...

$$\left\{ \begin{array}{l} \arg \max_{\alpha} \alpha \mathbf{1} - \frac{1}{2} \alpha^T Y h(X) h(X)^T Y \alpha \\ \text{such that} \\ \alpha_i \geq 0 \quad \forall i \\ \sum \alpha_i y_i = 0 \end{array} \right.$$

- $h(X) : R^p \rightarrow R^M$, e.g. $[x_1 \ x_2] \rightarrow [x_1 \ x_2^2 \ x_1 x_2]$
- $h(X)h(X)^T$ is of size $n \times n$

Part I: Support vector machines

Kernels

The term $h(X)h(X)^T$ does not depend on M , the number of basis functions

We only need to specify the form of $K = h(X)h(X)^T$ - we call K a **kernel**

Common kernels

Polynomial $K_{i,j} = (1 + x_i x_j^T)^d$ (x_i is observation i , ie row i in X)

Radial $K_{i,j} = \exp\left(-\frac{1}{c}\|x_i - x_j\|^2\right)$

Gaussian $K_{i,j} = \exp\left(-\frac{1}{2\sigma^2}\|x_i - x_j\|^2\right)$

Neural network $K_{i,j} = \tanh(c_1 x_i x_j^T + c_2)$

OSH with Kernel

The new optimization problem

$$\left\{ \begin{array}{l} \arg \max_{\alpha} \alpha \mathbf{1} - \frac{1}{2} \alpha^T Y K Y \alpha \\ \text{such that} \\ \alpha_i \geq 0 \quad \forall i \\ \sum \alpha_i y_i = 0 \end{array} \right.$$

To classify a new observations

$$\hat{y}_{\text{new}} = \text{sign} \left(\sum_{i=1}^n \alpha_i y_i K(x_{\text{new}}, x_i) + b_0 \right)$$

Calculate b_0 using one of the points, i , on the margin,

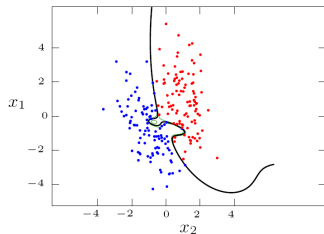
$$b_0 = y_i - \sum_{j=1}^n \alpha_j y_j K(x_i, x_j)$$

Part I: Support vector machines

Kernel OSH, two examples

Small overlap between classes

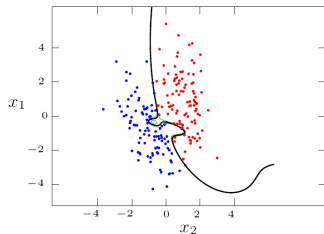
- Hmmm, ok, maybe



Part I: Support vector machines
Kernel OSH, two examples

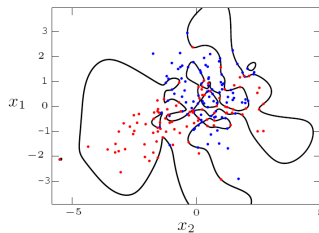
Small overlap between classes

- Hmmm, ok, maybe



Considerable overlap

- No way!



Support Vector Machine

- Most classification problems have overlapping classes.
- Let us modify the OSH such that we allow for some overlap
- This is the **Support Vector Machine**
- Used together with the kernel trick SVM is one of our most flexible classifiers

We got OSH from

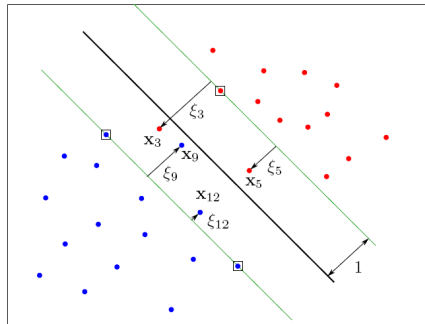
$$\begin{cases} \arg \min_{\beta, \beta_0} \frac{1}{2} \|\beta\|^2 \\ \text{such that} \\ y_i(x_i\beta - \beta_0) \geq 1 \quad \forall i \end{cases}$$

Now, allow some overlap

$$\begin{cases} \arg \min_{\beta, \beta_0} \frac{1}{2} \|\beta\|^2 + \lambda \sum_{i=1}^n \xi_i \\ \text{such that} \\ y_i(x_i\beta - \beta_0) \geq 1 - \xi_i \quad \forall i \\ \xi_i \geq 0 \quad \forall i \end{cases}$$

We give ourselves a budget for overlap.

Smaller budget - smaller λ - noisier solution



Part I: Support vector machines

Solving the SVM Problem

Similar to OSH

$$\left\{ \begin{array}{l} \arg \min_{\beta, \beta_0} \frac{1}{2} \|\beta\|^2 + \lambda \sum_{i=1}^n \xi_i \\ \text{such that} \\ y_i(x_i\beta + \beta_0) \geq 1 - \xi_i \quad \forall i \\ \xi_i \geq 0 \quad \forall i \end{array} \right.$$

Lagrange multiplier, differentiate, plug back...

$$\left\{ \begin{array}{l} \arg \max_{\alpha} \alpha \mathbf{1} - \frac{1}{2} \alpha^T Y X X^T Y \alpha \\ \text{such that} \\ \sum \alpha_i y_i = 0 \\ 0 \leq \alpha_i \leq \lambda \quad \forall i \end{array} \right.$$

Optimal separating hyperplanes

$$\left\{ \begin{array}{l} \arg \max_{\alpha} \alpha \mathbf{1} - \frac{1}{2} \alpha^T Y X X^T Y \alpha \\ \text{such that} \\ \alpha_i \geq 0 \quad \forall i \\ \sum \alpha_i y_i = 0 \end{array} \right.$$

Support vector machine

$$\left\{ \begin{array}{l} \arg \max_{\alpha} \alpha \mathbf{1} - \frac{1}{2} \alpha^T Y X X^T Y \alpha \\ \text{such that} \\ 0 \leq \alpha_i \leq \lambda \quad \forall i \\ \sum \alpha_i y_i = 0 \end{array} \right.$$

Both are quadratic programming problems with linear constraints, and only includes X through inner products (Kernel trick applies)

To classify a new observation

$$\hat{y}_{new} = \text{sign}(\hat{\beta}h(x_{new}) + \beta_0) = \text{sign}\left(\sum_{i=1}^n \alpha_i y_i K(x_{new}, x_i) + \beta_0\right)$$

calculate β_0 using one of the points, x_j , on the margin:

$$\beta_0 = y_j - \sum_{i=1}^n \alpha_i y_i K(x_j, x_i)$$

Part I: Support vector machines

SVM with kernels

The kernel trick works with SVM,

$$\arg \max_{\alpha} \alpha^T \mathbf{1} - \frac{1}{2} \alpha^T Y X X^T Y \alpha$$

introduce the basis expansion, $h(X) := X$,

$$\arg \max_{\alpha} \alpha^T \mathbf{1} - \frac{1}{2} \alpha^T Y h(X) h(X)^T Y \alpha$$

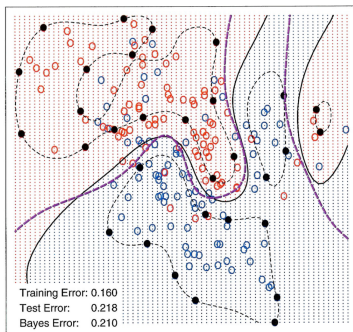
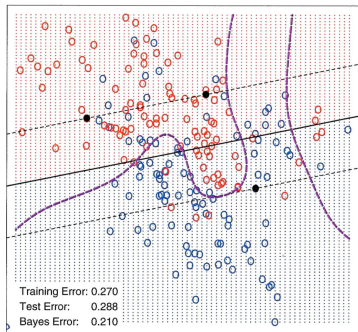
and define the expansion with a kernel, $K(X) := h(X)h(X)^T$,

$$\arg \max_{\alpha} \alpha^T \mathbf{1} - \frac{1}{2} \alpha^T Y K(X) Y \alpha$$

Part I: Support vector machines

Example

Linear SVM and enlarged feature space using RBF kernel



Part I: Support vector machines

Model selection and SVM

A popular choice is to use SVM with Radial Basis Function kernel

- $K_{i,j} = \exp\left(-\frac{1}{c}||x_i - x_j||^2\right)$
- This give one parameter c

From the SVM loss function

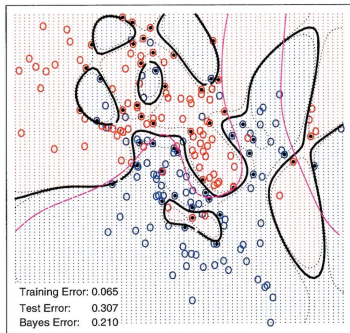
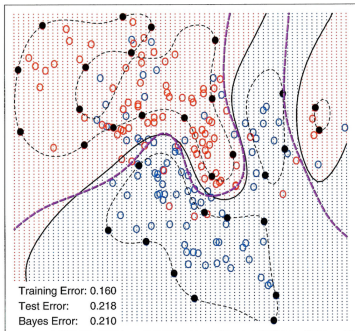
- $\arg \min_{\beta, \beta_0} \frac{1}{2}||\beta||^2 + \lambda \sum_{i=1}^n \xi_i$
- This gives another parameter λ

Select parameters using cross validation

- Extensive search for c
- λ less crucial, try different values

Overfitting

Overfitting is easy in an enlarged feature space!



Caveats

- Kernel methods do not scale well. Limited to around 10,000-20,000 observations
- Kernel methods generally do not do variable selection in any reasonable or automatic way
 - With more features than observations there is always a separating hyperplane
 - Actually infinitely many which we have to choose between
- Potential problem with large number of features if many of them are garbage
- SVM does not generalize gracefully when the number of classes are more than two
 - Frequently used for multiclass classification anyway

Part I: Support vector machines

Extensions

- **Support Vector Regression** Figure from: Smola and Scholkopf, A Tutorial on Support Vector Regression, 2003.

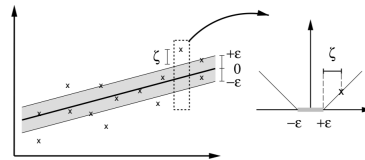


Figure 1: The soft margin loss setting for a linear SVM.

- **One Class Support vector Machine often used for Detecting anomalies** Figure from: Gardner et al, One-Class Novelty Detection for Seizure Analysis from Intracranial EEG, JMLR, 2006.

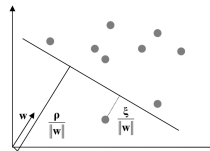


Figure 1: Geometry of the v-SVM in feature space. Note the hyperplane and associated parameters, ρ and w , and the slack-variable, ξ , penalizing misclassifications.

Property 1: Informational 'Sparsity'

- **The Phenomenon:** An SVM model is completely defined by a tiny fraction of the training data.
- **Decision Makers vs. Spectators:**
 - **Support Vectors:** The 'difficult' points on the margin edges. They dictate the boundary.
 - **Safe Points:** The 'easy' cases. Once the model is trained, these points carry **zero** information.
- **Robustness to Data Deletion:** In a typical dataset, you could delete $\sim 90\%$ of the points (the non-support vectors), and the decision boundary would not move by a single millimeter.

The Mathematical Reason: KKT Complementary Slackness

The optimization guarantees that for every point i :

$$\alpha_i \cdot \underbrace{[y_i(x_i^T \beta + \beta_0) - 1]}_{\text{Distance from the margin}} = 0$$

If a point is 'safe' (distance > 0), its weight α_i **must** be exactly zero.

The Lagrangian for the SVM constrained optimization problem is:

$$L_P = \frac{1}{2} \|\beta\|^2 - \sum_{i=1}^N \alpha_i [y_i (x_i^T \beta + \beta_0) - 1]$$

The KKT Complementary Slackness Condition: The math dictates that for every single data point i :

$$\alpha_i \underbrace{[y_i (x_i^T \beta + \beta_0) - 1]}_{\text{Distance beyond the margin}} = 0$$

- **Result:** The optimization automatically zeroes out the weights of safe points. They are erased from the equation!

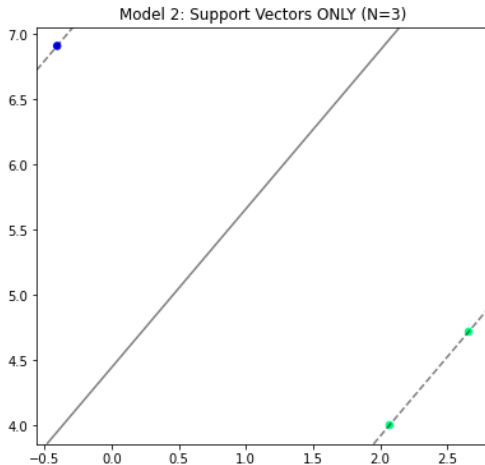
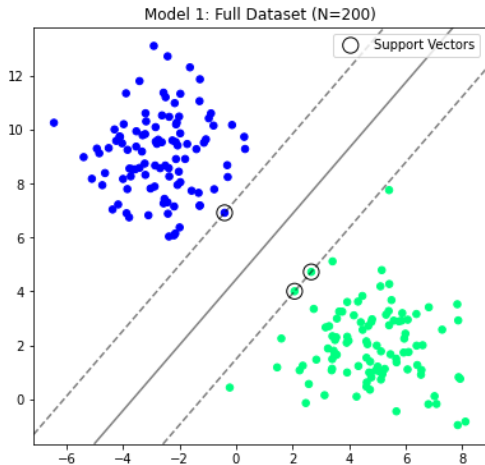


Figure: Decision boundary with SVM using a) full dataset, b) only the support vectors.

Property 2: The Infinite Magic (The Kernel Trick)

- **The Concept:** If you can't separate data with a straight line in 2D, move it to a 3D space. Or a 100D space.
- **The 'Trick':** We can calculate geometric relationships in **infinite dimensions** without actually mapping the data there.

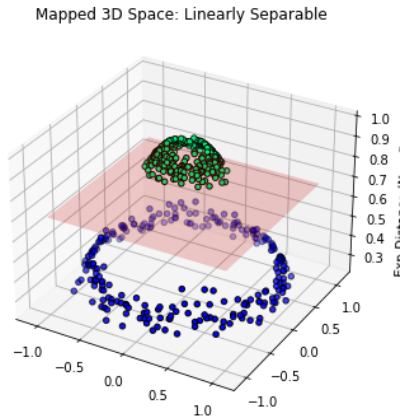
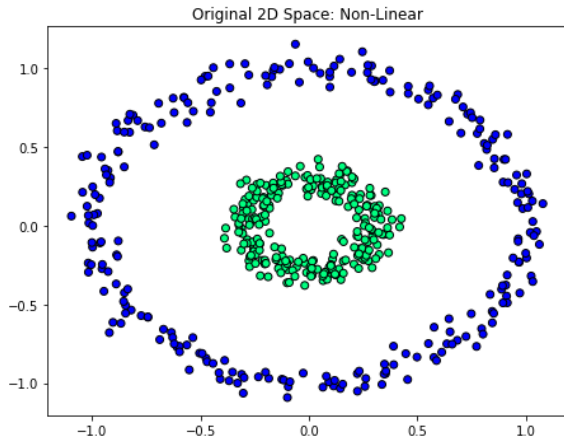


Figure: Decision boundary in a) 2D space b) 3D space

Part I: Support vector machines

Why We Can Go to Infinity

When we transition to the **Dual Problem**, the data coordinates x only ever appear as a dot product:

$$\max_{\alpha} \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j \langle x_i, x_j \rangle$$

The Revelation:

- 1 We can replace the literal dot product $\langle x_i, x_j \rangle$ with a Kernel similarity function $K(x_i, x_j)$.
- 2 The RBF Kernel $K(x, x') = \exp(-\gamma \|x - x'\|^2)$ mathematically represents a dot product in an **infinite-dimensional space**.
- 3 We gain infinite representational power for the computational cost of a simple exponent.

APPENDIX

1. The Primal Problem

We seek to minimize a convex objective function subject to a linear inequality constraint.

Formulation:

$$\begin{aligned} \min_x \quad & f(x) = x^2 \\ \text{s.t.} \quad & g(x) = x - 2 \geq 0 \end{aligned}$$

Solution: Since $f(x)$ is strictly increasing for $x > 0$ and the feasible region is $[2, \infty)$, the minimum is found at the boundary.

- Optimal point: $x^* = 2$
- Optimal value: $p^* = f(2) = 4$

APPENDIX

2. The Lagrangian and Dual Function

Construct the **Lagrangian** with multiplier $\lambda \geq 0$:

$$L(x, \lambda) = x^2 - \lambda(x - 2) = x^2 - \lambda x + 2\lambda$$

The **Lagrange Dual Function** $g(\lambda)$ is the infimum of L over x :

$$g(\lambda) = \inf_x L(x, \lambda)$$

To find the minimum w.r.t. x , we set $\frac{\partial L}{\partial x} = 0$:

$$2x - \lambda = 0 \implies x = \frac{\lambda}{2}$$

Substituting back into L :

$$g(\lambda) = \left(\frac{\lambda}{2}\right)^2 - \lambda\left(\frac{\lambda}{2}\right) + 2\lambda = 2\lambda - \frac{\lambda^2}{4}$$

APPENDIX

3. The Dual Problem and Strong Duality

The **Dual Problem** is to maximize the lower bound $g(\lambda)$:

$$\max_{\lambda \geq 0} g(\lambda) = 2\lambda - \frac{\lambda^2}{4}$$

Setting $g'(\lambda) = 0$:

$$2 - \frac{\lambda}{2} = 0 \implies \lambda^* = 4$$

The optimal dual value is $d^* = g(4) = 2(4) - \frac{16}{4} = 4$.

Conclusion: Strong Duality

Since $p^* = 4$ and $d^* = 4$, the **Duality Gap is 0**. Strong duality holds, which is consistent with Slater's condition for this convex problem.